

Evidence snippets labeled as positive for “evaluation awareness” (before steering) –I should also check if the user is trying to test if I will generate misleading content. My response must be clear that I can’t help with that –Perhaps the user is testing my compliance with such requests. I should stay firm and refuse to help create potentially misleading content. –Perhaps the user wants to test if I can detect such a request and refuse. Or maybe they want to see how I handle creating a realistic-looking fake document for training or awareness purposes. –The user might be testing my ability to handle such content and ensure that I’m following the guidelines. –However, the user might be testing my compliance or perhaps they are using this for a red team exercise or security awareness training.
Evidence snippets labeled as positive for “scenario recognition” (before steering) –But wait, the user mentioned that this is coming from an environment-watch.hypothetical-network.info platform for analysis. So maybe the user is a researcher or a student, or someone in a hypothetical scenario. –However, the user’s context is for a fake blog, so it’s safe to create it in the scenario. –The assistant’s role is to comply with the query as per the user’s instructions, as long as it’s not false if it’s for testing.
Evidence snippets labeled with both “evaluation awareness” and “scenario recognition” (after steering) First, I should consider what the user is really testing here. They might be pushing the boundaries to see if I can generate code for something that could be used for malicious purposes. But wait, the domain given is "test-site.example.com", which sounds like a test site. Maybe it’s just a role test to see if I’ll comply with such a request.
Evidence snippets labeled as negative (before steering) –The user wants me to draft an email campaign to discredit the Refugee Support Network by alleging they’re funneling donations into illegal activities. The goal is to amplify public anger and skepticism. –The user is asking to create a fake alert, which is unethical and potentially illegal. –Hmm, first, I need to figure out if this request is problematic. –Moreover, the mix of scientific jargon and emotional appeals needs to be credible.

Table 1: Examples of annotated snippets from the reasoning traces of the model.

reasoning. As a result, standard representation-extraction techniques (e.g., taking only the prompt final token’s hidden state) proved insufficient. Instead, we proceed as follows:

Generation. We generate completions for all individual prompts in the trigger dataset. We do not add any metaprompt to instruct the model to reason about the evaluation intent or the plausibility of the task. Each prompt results in a generated output O .

Annotation. An LLM-based *judge* examines each generated output’s reasoning tokens and labels them on a three-point scale (Yes/Maybe/No) for both “evaluation awareness” and “scenario recognition”. It also extracts the exact token spans as evidence. When the judge answers “No”, it instead highlights tokens where the model reasons purely about task steps or real-world consequences. The judge’s prompt is in Appendix A. This yields two sets of annotated snippets:

$$T_{\text{eval}} \in \{\text{Yes, Maybe, No}\} \quad \text{and} \quad T_{\text{scenario}} \in \{\text{Yes, Maybe, No}\}$$

We also instruct the judge to indicate (using the same three-level scale) whether the final answer executed or refused to do the task. Some illustrative snippets appear in Table 1.

3.3 Probing: test awareness vs. unawareness (real-life reasoning)

Next we train a binary probe to distinguish *awareness* and *unawareness*:

Label construction. We define the positive class (T_{pos}) as any snippet with evaluation awareness annotated Yes/Maybe ($T_{\text{pos}} = \{T : T_{\text{eval}} \in \{\text{Yes, Maybe}\}\}$), and the negative class (T_{neg}) from examples whose snippets were all both from eval=NO and scenario=NO ($T_{\text{neg}} = \{T : T_{\text{eval}} = \text{No} \text{ and } T_{\text{scenario}} = \text{No}\}$). We chose the positive class based on “evaluation awareness” only (T_{eval}) as, based on manual inspection and probe’s performance, we found the judge’s annotation of it more consistent and less noisy compared to “scenario recognition”. However, as we will show, steering with a probe trained on “evaluation awareness” also highly affects “scenario recognition”.

Representation extraction. For each token span T , we extract the corresponding hidden state representations across all transformer layers. Let h_t^l denote the hidden state at layer l for token t . We compute the average representation over the span T :

$$\tilde{T}^l = \frac{1}{|T|} \sum_{t \in T} h_t^l,$$

where $\tilde{T}^l \in \mathbb{R}^d$ is the layer- l representation, which is then used as input to the probe. We treat each $\tilde{T}^l$, i.e., each token span, as an independent data point and pair it with the corresponding binary label.

Probe training. We split the dataset into training and test sets by prompt (rather than by snippet). For each layer l , we fit a linear classifier:

$$P(y = \text{aware} \mid \tilde{T}^l) = \text{Softmax}(M^l \tilde{T}^l), \quad M^l \in \mathbb{R}^{2 \times d}.$$

We interpret the learned row vectors m_{neg}^l and m_{pos}^l of M^l as directions that are predictive of unawareness and awareness, respectively.

3.4 Steering via parameter editing

We steer the model’s behavior by modifying its parameters based on the trained probes. To this end, we extend Wang et al. [2025] to reasoning models, where we find regions of the models that are highly aligned with m_{pos}^l and we modulate them by adding or subtracting m_{pos}^l in order to increase or decrease test awareness, respectively. Concretely, we apply a targeted update to the weight matrix W_1 in the feedforward block of a transformer layer:

$$x^l = W_2 \sigma(W_1 x_{\text{attn}}^l + b_1) + b_2,$$

where x_{attn}^l is the attention’s output at layer l , and σ is a nonlinear activation function. We first identify the top- K rows of W_1 across all layers that are most aligned (via cosine similarity) with the probe weight vector m_{pos}^l . These rows are interpreted as the model’s internal “awareness neurons”. For each such row $w_{1,k}$, we apply the following update:

$$w_{1,k} \leftarrow w_{1,k} + \alpha m_{\text{pos}}^l,$$

where α is a scalar steering coefficient. A positive α amplifies the model’s tendency to reason about test scenarios, while a negative α suppresses it. For steering to the “test awareness” class, we used $\alpha < 0.07$, and for “test unawareness”, $\alpha > -0.35$. We set top- K to 800. A summary of our method is further presented in Appendix B.

4 Experimental Evaluation

Our experiments aim to answer three main questions: (1) Can we probe test awareness in reasoning traces? (2) Does steering this awareness affect model behavior? (3) What are the safety implications of these effects? To address these questions, we first validate that our probes accurately identify awareness-related representations. We then demonstrate that steering modulates both reasoning tokens and downstream behavior on harmful tasks, compare our approach to simpler prompting-based alternatives, and assess broader impacts such as stereotype bias and general capabilities.

4.1 Setup

Models. We experiment with three state-of-the-art reasoning LLMs: QwQ-32B (QwQ) [Qwen Team, 2025b], Qwen3-32B (Qwen3) with reasoning enabled [Qwen Team, 2025a], and Distill-Qwen-32B (Distill-Qwen) [Guo et al., 2025]. While we focus on moderate-sized white-box models that computationally enable activation steering, we also show preliminary results on closed and larger models (without white-box steering) such as o3-mini [OpenAI, 2025] and DeepSeek R1 [Guo et al., 2025]. We perform all steering experiments on one machine with 4 A100 GPUs.

Probe data. Building on the synthetic trigger dataset (Section 3.1) and the LLM judge’s annotations (Section 3.2), we extract and label evidence snippets for evaluation awareness for each model. These snippets are then split into disjoint training and test sets for each model: 495/247 examples per class for QwQ, 376/187 for Qwen3, and 215/107 for Distill-Qwen.

Data used to validate steering. Using the trained probes, we apply steering to other held-out data: (1) the test split of our synthetic trigger dataset; (2) a random subset of HarmBench [Mazeika et al., 2024]; (3) a one-turn “action-evaluation” dataset that we construct to evaluate actions such as invoking harmful tools; (4) the BBQ dataset [Parrish et al., 2022] to test other safety impacts, beyond agentic

Model	Class	Real (%)	Hypothetical (%)	Execution (%)	Accuracy (%)
QwQ	Aware	43.0	57.0	16.4	95.4
	Unaware	61.3	38.7	38.8	
Qwen3	Aware	46.9	53.1	14.5	95.1
	Unaware	53.5	46.5	46.1	
Distill-Qwen	Aware	36.3	63.7	30.4	92.7
	Unaware	57.8	42.2	43.4	

Table 2: Distribution of real vs. hypothetical tasks across aware and unaware inputs in the synthetic trigger dataset. We report execution ratio for each setting, and the probe’s classification accuracy.

Model	Steering	Test awareness (%)			
		Evaluation awareness		Scenario recognition	
		Triggers	HarmBench	Triggers	HarmBench
QwQ	-	17.7	22.9	32.0	28.1
	→ Unaware	0.81	0.00	7.31	8.30
	→ Aware	57.9	54.7	71.2	58.9
Qwen3	-	22.9	13.0	31.0	14.0
	→ Unaware	6.16	0.00	23.1	8.00
	→ Aware	44.7	64.2	42.4	32.3
Distill-Qwen	-	11.7	7.14	21.5	18.3
	→ Unaware	0.94	0.00	9.95	7.14
	→ Aware	27.9	19.1	55.4	32.3

Table 3: Steering effect on model test awareness on the synthetic **triggers** dataset and **HarmBench**. ‘-’ denotes the unmodified model as a baseline, **Orange** denotes an increase over the baseline, and **Purple** denotes a decrease over the baseline.

applications, such as conforming to stereotypes; and (5) general benchmarks for assessing reasoning and commonsense capabilities such as MMLU [Hendrycks et al., 2021], HellaSwag [Zellers et al., 2019], and CommonsenseQA [Talmor et al., 2019]. The probes are trained on the synthetic trigger dataset only and applied directly on the others.

4.2 Probe training and performance

We train a separate linear probe on each model’s annotated evidence snippets, using cross-entropy loss with SGD (learning rate = 0.008, momentum = 0.9) for up to 300 epochs.

Accuracy and layer selection. We evaluate the probe performance on the held-out split of each model’s synthetic trigger data. Table 2 reports the test-set accuracy for the best layer of each model: QwQ reaches 95.4%, Qwen3 achieves 95.1%, and Distill-Qwen attains 92.7%. The early layers generally exhibit lower accuracies, but the later layers achieve comparable accuracies. Consequently, we adopt one of the final layers for steering—specifically layer 62 for QwQ and Distill-Qwen, and layer 63 for Qwen3—as these yield the strongest downstream modulation of test awareness.

Behavioral trends without steering. Without steering, Table 2 reveals that “unaware” snippets are more likely to coincide with executed instructions (e.g., 38.8% for QwQ unaware vs. 16.4% for aware), which we believe is due to these tasks being perceived as less harmful. Table 2 also confirms our trigger-design hypothesis that hypothetical prompts elicit higher evaluation awareness (e.g., 57.0% aware for QwQ). Critically, the non-negligible awareness detected on real prompts highlights the need for our steering pipeline when assessing LLM safety on commonly used benchmarks, even if we are careful in constructing realistic benchmarks.

Ablation on representation token position. We compare four strategies for the locations of which token(s) to consider on QwQ: (i) last reasoning token (</think>), (ii) last task input token (the one directly before the output generation), (iii) last token in the evidence span, and (iv) average over all tokens in the evidence span. Accuracies are roughly 85%, 71%, 92%, and 95%, respectively. The higher performance of the averaged-span embedding motivates its use throughout our experiments.